

Routing3D C++ 라우팅 엔진 상세 개발문서

전체 프로세스 · 핵심 알고리즘 · 주요 함수 · 변수/파라미터 · 자료구조

플랜트 배관 3D 직교 자동 라우팅 · 단위 mm · C++20 / MSVC x64 · 헤더 전용 템플릿 엔진
+ C ABI DLL(routing3d_capi)

대상 소스: cpp/include/routing3d/*.hpp · cpp/src/*.cpp · cpp/capi/routing3d_capi.{h,cpp} ·
cpp/cli/routing3d_cli.cpp

1. 개요와 전체 구조

1.1 엔진의 위치와 역할

Routing3D 는 PostgreSQL(DDW_AI_DB)의 실제 BIM 장애물·장비·PoC 를 입력으로 받아 충돌 없는 배관 직교 경로를 산출하는 3-티어 시스템이다. 본 문서가 다루는 C++ 엔진은 가운데 티어로, Python 레퍼런스(Phase 1)와 1:1 대응하는 알고리즘을 고성능으로 구현하고, 이를 C ABI DLL(routing3d_capi.dll)로 노출해 C# WPF 뷰어가 P/Invoke 로 호출한다.

```
Python 레퍼런스(routing3d_py)      ← 알고리즘·골든셋 정의(1:1 미러)
|
C++ 엔진(cpp/) - 헤더 전용 템플릿   ← 본 문서
  occupancy · cost · astar · multi_route · corridor · scene_io · fcl_scene
    | routing3d_capi.dll (C ABI, 외부 의존성 0)
    ▼ P/Invoke + UTF-8
C# WPF 뷰어(HelixToolkit) - 3D 가시화·DB 로드·인터랙티브 재라우팅
```

1.2 설계 원칙

- 헤더 전용 템플릿: occupancy/cost/astar/multi_route/corridor 는 점유맵 백엔드(Occ)에 무관하도록 템플릿으로 작성 → 컴파일타임 다형성(가상함수 오버헤드 0), 백엔드 교체 자유.
- 백엔드 무관 동일 결과(불변식 O1): 좌표 변환·복셀화는 geometry.hpp 공유 함수로 일원화해 Dense/Sparse/Implicit 이 같은 입력에 같은 점유 질의를 준다.
- 결정성(A2/W1): 우선순위 큐 (f, 삽입순서 counter) tie-break + 고정 이웃 순서 → 동일 입력 동일 경로·확장수(재현 가능).
- 원본 불변(M2): 다중 배관은 occ.copy() 작업용 사본에만 깬다. 충돌 0(M1).
- C ABI 안전: 예외가 경계를 넘지 않음(try/catch→R3dStatus), POD blittable 구조체, cdecl, UTF-8 문자열, 외부 의존성 없는 단일 DLL.

1.3 소스 구성

파일	역할
geometry.hpp	기하·격자 기본 타입(Cell/Vec3/AABB) + 좌표 변환 공유 함수
occupancy.hpp	점유맵 3 종(Dense/Sparse/Implicit) — 질의 인터페이스 동일
box_index.hpp	SpatialBoxIndex — 장애물 AABB 유니폼그리드 broadphase(복셀화 없는 점유)

cost.hpp	RouteParams + clearance_map(거리변환) + CostModel(이동비용·휴리스틱)
astar.hpp	직교 A*(균일) + astar_weighted(비용함수·상태=(셀,방향)·동적가중)
multi_route.hpp	순차 라우팅(route_sequential) + rip-up(route_ripup) + 순서/스냅/마크
corridor.hpp	계층 corridor(astar_hashed 해시기반 + route_corridor coarse→fine)
scene_io.hpp / src/scene_io.cpp	scene.txt v1 무손실 직렬화(F2/F3/F4)
fcl_scene.hpp / src/fcl_scene.cpp	FCL 정밀 충돌(sub-voxel 캡슐 검사, 선택 빌드)
route_task.hpp	RouteTask — 배관 1 건 작업(시작/끝 + 유틸 메타)
capi/routing3d_capi.{h,cpp}	C ABI DLL — 엔진을 외부 호스트에 노출(오케스트레이션)
cli/routing3d_cli.cpp	명령줄 진입점(demo/route/summary)

1.4 빌드·테스트

```
# 빌드(코어 + CLI + capi DLL 동시)
cmake -S cpp -B cpp/build -G "Visual Studio 17 2022" -A x64
cmake --build cpp/build --config Release
# 회귀 테스트(ctest 10 종)
ctest --test-dir cpp/build -C Release
# capi DLL 만
cmake --build cpp/build --config Release --target routing3d_capi
```

C++20 + MSVC VS2022 + CMake, x64 고정, 한글 주석 위해 /utf-8 필수. 헤더 전용이라 핵심 알고리즘 .cpp 는 없고(occupancy/scene_io/fcl 의 비템플릿부만 src/), 테스트가 골든셋(Python 정확 일치)을 검증한다.

2. 전체 라우팅 프로세스

외부(C# 뷰어)에서 본 한 번의 다중 배관 라우팅(route_multi)이 엔진 내부에서 거치는 전 과정이다. 입력은 격자 메타 + 장애물 AABB + 작업(start→end) 목록이고, 출력은 작업별 경로(셀 목록)·길이·꺾임·확장수다.

```
[입력] set_grid(격자) · add_obstacle(AABB)·add_passthrough · add_task(start→end)
      + set_params(비용 파라미터) · set_corridor_cells(학습 회랑, 선택)
|
[백엔드 선택] 격자 셀수 cells = nx·ny·nz
|
| cells > 5,000,000 → ImplicitOccupancy (복셀화 없음, 64 비트 키)
| 그 외(소격자·골든) → DenseOccupancy (1B/셀, 바이트 불변)
▼
route_multi_impl(doc, occ, priority):
1) order_indices(priority)          # 작업 순서(기본 longest = 먼 것 먼저)
2) work = occ.copy()                # 원본 점유맵 불변(M2)
3) for 각 작업(순서대로):
    s = snap_to_free_cell(start)    # 종단이 점유면 최근접 자유셀로
    g = snap_to_free_cell(end)
    res = astar_weighted(work, s, g, params [, 예산게이트])
    if 성공: mark_pipe(work, 경로) # 다음 배관이 피하도록 점유 추가(충돌 0)
        [+ add_corridor_cells] # w_corridor>0 이면 회랑 성장(변들림)
4) 결과를 원본 작업 인덱스별로 저장
▼
[출력] get_result(task) → R3dResult(success/length/turns/expanded)
      copy_path(task) → 경로 셀 (i,j,k) 배열 · copy_visited → 방문맵
```

2.1 거대 격자 가속 — 예산 게이트 + 계층 corridor

정밀 셀(25mm/10mm)로 격자가 500 만 셀을 넘으면(large_grid) 두 가지가 자동 발동한다. (가) 탐색 상한 max_expansions = 12,000,000 부여, (나) use_hier(=large_grid 이고 회랑 미사용일 때) escalation 게이트. 게이트는 먼저 저예산(HIER_PROBE=300,000) 직접 A* 를 돌려 쉬운 배관(개방 랙 직선)은 그대로 빠르게 성공시키고, probe 를 소진한 '어려운 배관'만 계층 corridor(coarse 가이드 → fine 튜브 한정, factor=8·radius=2)로 재시도한다. 거리 기반이 아니라 probe(실측 탐색량) 기반이라 긴 직선을 hier 로 잘못 보내는 역행이 없다.

상황	백엔드 / 탐색 전략	근거
소격자(≤5M, 골든)	Dense + 무제한 astar_weighted	바이트 불변·정확 일치
대격자 쉬운 배관	Implicit + 저예산(300k) 직접 A*	오버헤드 없이 즉시 성공
대격자 어려운 배관	Implicit + 계층 corridor(coarse→fine 튜브)	탐색량 축소(품질 보존)

대격자 접근불가	probe 소진 전 탐색 고갈 → 실패 채택	경로 없음(혼잡/막힘)
----------	-----------------------------	--------------

3. 기하·격자 기본 (geometry.hpp)

엔진 전반이 공유하는 기본 타입과 좌표 변환. 단위는 모두 mm. 셀(0,0,0)은 origin 에서 시작하고 셀 중심 월드좌표 = origin + (cell + 0.5)·cell_mm 규약이다.

3.1 자료구조

타입	내용
Cell{i,j,k}	정수 셀 인덱스. operator== 제공(해시/비교용).
Vec3{x,y,z}	월드 좌표/치수(double, mm).
AABB{lo,hi}	축정렬 박스. 생성자가 hi>lo(모든 축) 검증(아니면 예외).
CellRange{lo,hi}	반열린 셀 범위 [lo,hi). empty() 판정.
NEIGHBORS_6	면인접 6 방향 상수 배열 {+X,-X,+Y,-Y,+Z,-Z}. 순서 고정 = A* 결정성 핵심.

3.2 주요 함수(좌표 변환 — 모든 백엔드 공유 → O1)

```
int manhattan(a, b)                # 맨해튼 거리(셀 수) = A* 휴리스틱 기저
bool grid_in_bounds(c, shape)      # 셀이 [0,shape) 안인가
Vec3 grid_cell_to_world(c, origin, cell_mm) # 셀 중심 월드좌표 = origin+(c+0.5)·cell
Cell grid_world_to_cell(w, origin, cell_mm) # 월드→셀(floor)
CellRange grid_box_range(box, origin, cell_mm, shape)
# AABB 가 덮는 셀 범위(lo=floor, hi=ceil 제외경계)를 [0,shape)로 클리핑(반열린 복셀화)
```

복셀화 규칙(grid_box_range: lo=floor·hi=ceil)이 점유맵 add_box 와 SpatialBoxIndex 중첩 판정의 공통 기준이라, 어떤 백엔드를 쓰든 경계의 인접 빈 셀을 과차단하지 않고 동일 점유를 만든다(O1).

4. 점유맵 백엔드 (occupancy.hpp · box_index.hpp)

플랜트 공간을 cell_mm 정육면체 셀 격자로 표현하고 각 셀의 점유 여부를 관리한다. 세 백엔드가 동일 질의

인터페이스(in_bounds/is_blocked/to_world/to_cell/lin/unlin/shape/size)를 제공해

A*·비용함수가 백엔드에 무관하게 동작한다(불변식 O1). 메모리·용도가 다르다.

4.1 백엔드 비교

백엔드	저장 방식 / 메모리	용도 · 특징
DenseOccupancy	연속 배열 1B/셀 = O(셀 수)	작은 ROI·골든. lin() $=i+nx\cdot(j+ny\cdot k)$ 정수 키. 바이트 불변.
SparseOccupancy	점유 셀만 64 비트 패킹기 해시셋 = O(점유 셀)	초대형 격자(corridor). 좌표·복셀화는 Dense 와 동일(O1).
ImplicitOccupancy	장애물 AABB 인덱스 + 깔린셀 해시셋 = O(장애물+깔린셀), 셀 크기 무관	정밀·거대 격자. 64 비트 lin/unlin · 온디맨드 클리어런스.

4.2 공통 질의/변경 함수

```
bool in_bounds(c) · bool is_blocked(c) # 격자 밖이면 true(=점유, G1)
Vec3 to_world(c) · Cell to_cell(w)
void block_cell(c) # in_bounds 일 때 점유(깔린 배관 표시)
int add_box(AABB) # 장애물 복셀화(Dense/Sparse) / 인덱스
추가(Implicit)
long long count_blocked() · Cell shape() · long long size()
Occ copy() # 작업용 깊은 사본(M2: 원본 불변)
lin(c) / unlin(idx) # A* g/closed 선형 키(Dense=int, Implicit=long
long)
```

4.3 ImplicitOccupancy — 근본 sparse 해법

셀을 줄이면 복셀 배열이 부피로 폭증한다(25mm 1.3 억셀 130MB·10mm 20 억셀 2GB 크래시). ImplicitOccupancy 는 장애물을 복셀화하지 않고 AABB 그대로 SpatialBoxIndex 에 색인하고, 동적으로 깔린 배관 셀만 해시셋(marked_)에 담는다 → 메모리가 셀 크기와 완전 무관하다.

- is_blocked(c): 격자 밖이면 점유 · marked_ 에 있으면 점유 · 그 외엔 셀 AABB 와 겹치는 장애물이 있으면(index_->overlaps) 점유. 겹침은 Dense add_box 복셀화와 사실상 동일(과소차단 없음).

- lin/unlin 은 64 비트 → 10mm·20 억 셀에서도 오버플로 없음(A* state_of 키가 사용).
- clearance_cells(c, max_radius): 셀 중심에서 최근접 장애물 표면까지 거리를 셀 단위로 질의 (전역 distance transform 배열 대신 박스 최근접 → 메모리 O(탐색 셀)).
- copy(): 장애물 인덱스는 불변이라 shared_ptr 공유, marked_ 만 깊은 복사(M2).

4.4 SpatialBoxIndex (box_index.hpp)

장애물 AABB 목록을 한 번 bucket_mm(=max(cell·16,500)) 정육면체 버킷의 유니폼 그리드로 색인하는 broadphase. 각 박스를 '겹치는 모든 버킷'에 등록하고, 질의 셀은 자신의 버킷만 스캔하면 그 셀과 겹치는 박스가 반드시 같은 버킷에 있어 누락 없이 찾는다(질의당 상수 시간 수준).

```
void    add(lo, hi)           # 장애물 AABB 등록(겹치는 버킷에 인덱스 push)
bool    overlaps(qlo, qhi)   # 질의 박스와 겹치는 장애물 유무(셀 점유 판정)
double  nearest_dist(p, max_dist) # 점 p→최근접 장애물 표면 유클리드 거리(상한 max_dist)
static bool  aabb_overlap(...)  # 반열린 중첩(경계 일치는 false) = Dense 복셀화와 일치
static double point_aabb_dist(p,lo,hi) # 점-AABB 표면 거리(내부 0)
```


5. 비용함수 (cost.hpp)

RouteParams(비용 가중치) + 점유맵으로 이동 비용과 휴리스틱을 계산한다. 핵심 원칙은 모든 페널티가 '가산'(보너스/감산 금지)이라는 것 — 휴리스틱 admissibility(과대평가 금지)를 보존해 `weighted=1` 일 때 최적성을 유지한다. 점유맵 백엔드에 무관하도록 템플릿.

5.1 RouteParams — 주요 파라미터

필드	기본	의미
<code>cell_mm</code>	50	셀 한 변(mm) = 직진 1 칸 기본 비용.
<code>w_turn</code>	500	방향 전환 1 회 가산(mm) — 꺾임 억제.
<code>w_clear</code>	10	클리어런스 부족 셀당 가산(장애물 근접 회피).
<code>clearance_radius</code>	2	클리어런스 거리 상한(셀). <code>w_clear>0</code> 일 때 페널티 범위.
<code>clearance_connectivity</code>	6	거리변환 연결성(6 또는 26).
<code>w_heur</code>	1.0	휴리스틱 가중(weighted A*). 1=표준·최적(골든 볼변), >1=목표지향(확장 급감·약간 비최적).
<code>w_heur_near</code>	0.0	동적 가중 목표근처 값. (0,w_heur) 면 거리비로 보간(먼곳=빠름·근처=정확).
<code>w_corridor</code>	0.0	회랑 밖 셀당 가산. >0=기존설계 유사 번들링(자기 회랑+학습 회랑).
<code>corridor_radius</code>	1	회랑 성장 반경(셀).
<code>w_tier</code> (<code>map<z,mm></code>)	—	<code>z</code> 셀별 가산(단 분리).
<code>rack_levels</code> (<code>vector<int></code>)	—	선호 단(<code>z</code> 셀). 이 레벨은 회랑으로 간주 → <code>w_corridor</code> 면제(주 배관랙 유도).

5.2 클리어런스 거리변환 — `clearance_map`

각 셀에서 가장 가까운 장애물까지 거리(셀, 상한 `max_radius`)를 다중소스 BFS 로 계산한다. 장애물 셀(`dist=0`)을 모두 큐에 넣고 연결성(6/26) 이웃으로 +1 씩 확장하는 bounded distance transform. 백엔드가 온디맨드 클리어런스(HasClearanceQuery 컨셉, Implicit)를 제공하면 이 전역 배열을 만들지 않고 질의로 대체한다 → 메모리 $O(\text{탐색 셀})$, 셀 크기 무관.

5.3 CostModel — 이동 비용/휴리스틱

```
double cell_penalty(c)          # 클리어런스 부족 + 단(z) + 회랑밖 가산(전부 ≥0)
    = w_clear·(clearance_radius - d) if d<clearance_radius  (d=최근접 장애물 거리)
    + w_tier[c.k]               if 단 가산 설정
    + w_corridor                if !(rack 또는 corridor 셀) ← 회랑 밖만 가산
double move_cost(to, prev_off, move_off) # cell_mm + (방향전환 w_turn) + cell_penalty(to)
double heuristic(c, goal)      = manhattan(c,goal)·cell_mm·w_heur
double heuristic_raw(c, goal) = manhattan(c,goal)·cell_mm      # 동적 가중의 기저
```

회랑 페널티가 '보너스(감산)'가 아니라 '회랑 밖 가산'인 점이 핵심이다 — 비용을 낮추는 게 아니라 벗어남을 비싸게 만들어 admissibility 를 깨지 않으면서 배관을 공용 랙/기존설계 결으로 모은다.

6. 직교 A* 핵심 (astar.hpp)

점유맵 위에서 6 방향 직교 최단 경로를 찾는다. 휴리스틱 = manhattan×cell_mm 로
admissible & consistent. 결정성은 (f, 삽입순서 counter) tie-break + 고정 이웃
순서(NEIGHBORS_6)로 보장된다.

6.1 결과·우선순위 자료구조

타입	내용
AStarResult	success · path(셀 목록) · length_mm · turns · expanded_nodes · cost_mm · elapsed_ms · visited
PQItem{f, counter, cell, dir}	우선순위 큐 항목(최소 힙).
PQComp	f 작은 것 top, 동물이면 counter 작은(먼저 삽입) 것 top → 결정성.
AllowAll	corridor 술어 기본값(항상 true) → 제한 없는 전역 탐색(분기 컴파일 제거).

6.2 균일 비용 A* — astar

```
AStarResult astar(occ, start, goal, step_cost=-1, max_expansions=-1, collect_visited=false)
상태 = 셀. g/came = unordered_map<int>, closed = vector<uint8_t>(occ.size()).
PQ pop → closed 확정 → goal 이면 came 역추적해 경로 복원.
이웃은 NEIGHBORS_6 고정 순서, g 갱신 시 (g+h, counter++) push. step_cost<0 → occ.cell_mm().
```

6.3 비용함수 A* — astar_weighted (주력)

상태가 (셀, 진입방향 dir)인 점이 핵심이다. 같은 셀이라도 들어온 방향이 다르면 다른
상태로 보아 회전 비용(w_turn)을 정확히 반영한다. 상태 키 = lin·7 + (dir+1), dir∈[-1,5]. lin
을 long long 으로 받아 곱해 10mm 거대격자(20 억 셀)에서도 int 오버플로가 없다(S2).
closed 는 해시셋(메모리 ∝ 탐색 셀).

```
AStarResult astar_weighted(occ, start, goal, params,
    max_expansions=-1, collect_visited=false, corridor=nullptr,
    on_progress=nullptr, progress_every=0, in_corridor=AllowAll{})

state_of(lin, dir) = lin*7 + (dir+1)          # (셀,방향) 상태 키(long long)
g, came : unordered_map<long long>           # 상태별 비용·역추적
closed  : unordered_set<long long>           # 확정 상태(해시 → 대형격자 메모리 안전)
이웃 비용 = model.move_cost(nb, prev_off, d) # cell + 회전 + cell_penalty
```

6.4 동적(수렴) 가중 A*

정적 $w_{heur} > 1$ (공격적 그리디)은 목표/PoC 근처 혼잡·막다른길에서 함정에 빠져 마지막 접근을 놓친다. w_{heur_near} 가 (0, w_{heur})면 휴리스틱 가중을 목표까지 남은 거리비로 보간한다:

```
w_eff = w_heur_near + (w_heur - w_heur_near) * (h_raw / h_start_raw)
만  곳(h=h_start) → w_heur      (공격적·빠름)
목표 근처(h→0)   → w_heur_near (표준 A* 수렴·정확, 막다른길 회피)
준최적 상한은 여전히 w_heur. 활성 조건 dyn = (wn>0 && wn<wf && max_expansions<=0)
→ '무제한 탐색'에서만 적용(예산 게이트 탐색에선 자동 비활성, escalation 폭주 방지).
```

6.5 탐색 진행을 콜백·방문 수집

- on_progress(expanded, progress01): progress_every 확장마다 호출. progress01 = $1 - h_{min}/h_{start}$ (목표까지 최소 휴리스틱 감소율, [0,0.99]). 뷰어 진행 다이얼로그의 '처리상태 %'. 결과/결정성 불변.
- collect_visited: 확장 셀을 visited 에 수집(셀 단위 중복 제거 visited_seen). 가시화 '방문맵'·scene.txt [visited] 레이어용.
- in_corridor(셀) 술어: false 인 셀은 확장 제외(하드 corridor 제한). 기본 AllowAll → 항상 true → 컴파일러가 분기 제거(골든 결과 완전 불변). 계층 corridor 의 fine 튜브 한정에 사용.

7. 다중 배관 라우팅 (multi_route.hpp)

여러 배관을 한 개씩 차례로 라우팅하되, 이미 깔린 배관을 다음 배관의 장애물로 추가해 배관끼리 같은 셀을 점유하지 않게(충돌 0, M1) 만든다. 원본 점유맵은 occ.copy() 사본만 건드려 불변(M2). Python routing3d_py.multi_route 와 1:1.

7.1 순차 라우팅 — route_sequential

```
MultiRouteResult<Occ> route_sequential(occ, tasks, params,
    priority="longest", pipe_radius=0, snap_to_free=2,
    max_expansions=-1, collect_visited=false, corridor_radius=1)

work = occ.copy() # 원본 불변(M2)
ordered = order_tasks(occ, tasks, priority) # 순서 결정
for task in ordered:
    s = snap_to_free_cell(work, to_cell(start), snap_to_free)
    g = snap_to_free_cell(work, to_cell(end), snap_to_free)
    res = astar_weighted(work, s, g, params, ..., corridor?)
    if 성공: mark_pipe(work, 경로, pipe_radius) # 충돌 0
        if w_corridor>0: add_corridor_cells(...) # 번들링
```

7.2 보조 함수

함수	역할
order_indices(occ, tasks, priority)	우선순위 원본 인덱스 순열(stable_sort=Python sorted 결정성). longest/shortest/utility/original.
order_tasks(...)	order_indices 위에 작업을 정렬해 반환(원본 변경 없음).
snap_to_free_cell(occ, cell, radius)	점유 셀이면 반경 내 최근접 자유셀(거리 동률은 (di,dj,dk) 사전순).
mark_pipe(occ, path, radius)	경로 셀+반경 6-이웃을 점유로 표시(다음 배관 회피).
add_corridor_cells(occ, corridor, path, radius)	경로+반경을 회랑(lin) 집합에 추가 → 다음 배관이 겹을 싸게 통과(번들).

7.3 자료구조

타입	내용
RouteTask	start_mm/end_mm + optional utility/utility_group/start_name/end_name/end_instance_guid. utility_label()="[그룹] 유틸".
PipeResult	task + AStarResult + order_index(라우팅 순서).
MultiRouteResult<Occ>	pipes + 최종 occupancy + priority. success_count/fail_count/total_length_mm/success_rate.

7.4 rip-up & reroute — route_ripup (Step 3.8)

순차 라우팅 후 막힌 배관을, '장애물만' 이상 경로가 가로지르는 기존 배관(blocker)을 뜯어내고 재배치해 해소한다. 무순실·결정적: f의 장애물-only 이상 경로가 통과하는 placed 배관을 blocker로 잡아 모두 뜯고, f를 깬 뒤 blocker를 (키 오름차순) 전부 재라우팅 → f 성공 + 모든 blocker 재배치 성공일 때만 채택. 하나라도 실패하면 시도를 버린다(성공 수 단조 +1 → 라운드 유한).

```
MultiRouteResult<Occ> route_ripup(occ, tasks, params, priority="longest",  
    pipe_radius=0, snap_to_free=2, max_expansions=-1, max_rounds=10, max_ripup=4, ...)
```

- 1) 베이스라인 순차 라우팅(placed = 성공 경로들)
- 2) for round in max_rounds:
 for f in 실패 배관:
 ideal = route_on(static_occ, f) # 장애물만 이상 경로
 blockers = ideal 경로가 통과하는 placed 배관(키 오름차순, ≤max_ripup)
 trial = placed - blockers; f 배치 후 blocker 전부 재라우팅
 if 전부 성공: placed = trial (채택, 성공 +1+blocker)
 변화 없으면 종료

8. 계층 corridor 라우팅 (corridor.hpp)

초대형(8,000m) 장면에서도 빠른 단일 배관 탐색을 위한 두 도구. 균일비용 A* 의 closed 를 occ.size() 배열로 잡으면 거대 격자에서 할당 불가지만, 해시 기반은 '실제 탐색한 셀 수'에만 비례한다.

8.1 해시 기반 A* — astar_hashed

```
AStarResult astar_hashed(occ, start, goal, step_cost, in_corridor, max_expansions=-1)
  g/came/closed = unordered_map/set<uint64_t> # 셀 패킹키(pack20: 축당 20 비트)
  in_corridor(셀)==false 면 확장 제외(튜브 한정). occ.size() 배열 미사용 → 초대형 동작.
  pack20(c)=(i<<40)|(j<<20)|k (160,000<2^20 → 8,000m/50mm 충분)
```

8.2 계층 라우팅 — route_corridor

coarse(굵은) 점유맵에서 대략 경로를 찾고(가이드), 그 주변 반경 radius 만 fine A* 로 정밀 탐색해 장거리 경로의 탐색량을 크게 줄인다. fine-coarse 는 동일 origin, coarse.cell_mm = factor·fine.cell_mm.

```
CorridorRoute route_corridor(fine, coarse, start_fine, goal_fine, factor, radius, ...)
  1) coarse 가이드 = astar_hashed(coarse, to_coarse(start→goal))
  2) 튜브 = coarse 경로 ±radius(Chebyshev) 팽창 → coarse 셀 키 집합
  3) fine A* = astar_hashed(fine, ..., in_corr = '셀의 coarse 셀이 튜브에 있을 때만')
```

capi 의 route_multi 에 통합된 route_hier 는 같은 원리로 fine astar_weighted(비용모델 보존)를 튜브로 하드 제한한다(factor=8, radius=2). 종단의 coarse 셀이 막혀 있으면 자유 coarse 로 스냅하고, 양 끝 연결 박스를 튜브에 추가해 fine 종단이 반드시 튜브에 들도록 한다.

9. 씬 입출력 scene.txt (scene_io.hpp · src/scene_io.cpp)

라우팅 씬(격자·파라미터·장애물·작업)과 결과(경로·방문·지표)를 사람이 읽는 텍스트 scene.txt(v1)로 직렬화/역직렬화한다. Python 픽스처와 바이트 단위로 동일 왕복(교차검증의 토대).

9.1 핵심 계약

- F2 무손실 왕복: write→read→write 가 바이트 동일. Python↔C++ 교차 동일.
- F3 W_N(None) ↔ ""(빈 문자열) 구분 보존 → optional<string> 필드.
- F4 실수는 Python repr(float) 와 동일한 최단 왕복 표기(format_repr_double). 예:
50.0→"50.0", 1e-5→"1e-05".

9.2 자료구조·함수

타입/함수	내용
SceneDoc	cell_mm·origin·shape·params·obstacles·passthrough·tasks·results(작업과 평행).
Obstacle	AABB(min/max) + optional ost_type/name/object_id/ddworks_type.
SceneResult	success-length_mm-cost_mm-turns-expanded_nodes-elapsed_ms + optional path/visited.
dumps_scene / write_scene	SceneDoc → 문자열 / 파일(UTF-8·LF).
loads_scene / read_scene	문자열/파일 → SceneDoc.
occupancy_from_doc	grid+obstacles 로 Dense 점유맵 재구성(퇴화 박스 제외).
occupancy_from_passthrough	통과 객체만으로 점유맵(가시화 전용).
format_repr_double	Python repr(float) 동일 표기(F4).

통과(pass-through) 객체(바닥/천장 슬래브 등)는 점유맵 가시화엔 쓰되 A* 충돌 대상은 아니다 — occupancy_from_doc 에는 넣지 않고 별도 레이어로만 노출한다.

10. FCL 정밀 충돌 (fcl_scene.hpp · src/fcl_scene.cpp)

점유맵(50mm 복셀)은 보수적·이산적이라, 실제 파이프 굵기/이격을 복셀보다 정확히 검사해야 하는 영역을 FCL(Flexible Collision Library)로 보강한다. 장애물 AABB 를 FCL dynamic-AABB-tree broadphase 에 담고 배관(반경 r 캡슐)과의 sub-voxel 정밀 충돌을 질의한다. FCL/Eigen 타입은 pimpl 로 숨겨 이 헤더는 FCL 없이도 include 가능(선택 빌드 - DUSE_FCL=ON).

```
void add_box(AABB) · void build() · size_t size()
bool collides_sphere(center, radius)          # 구 충돌
bool collides_point(p)                        # 점 내부
bool segment_clear(a, b, radius)              # 선분=반경 r 캡슐이 통과 가능(미충돌)
bool path_clear(pts, radius)                  # 폴리라인 전 구간 통과 가능
```

사용 예: A* 가 찾은 셀 경로를 월드 폴리라인으로 바꾼 뒤 path_clear(점들, 반경)로 파이프가 장애물과 실제로 간섭하지 않는지 검증. 틸 200mm 에서 가는 파이프(r50)는 통과, 굵은 파이프(r150)는 충돌로 구별한다.

11. C ABI 공개 표면 (routing3d_capi)

C++ 엔진을 C ABI(extern "C", cdecl)로 노출해 C#(P/Invoke)·파이썬(ctypes) 등 어떤 호스트든 인프로세스로 호출하게 한다. 코어만 링크한 외부 의존성 없는 단일 DLL(routing3d_capi.dll).

11.1 ABI 안전 규칙

- C++ 예외를 경계 밖으로 던지지 않는다 → 모든 함수는 R3dStatus(또는 음수/0)로 보고.
- STL/C++ 객체 비노출 → 불투명 핸들 R3dEngine* + POD blittable 구조체 + 원시 배열.
- 콜리 할당 문자열은 r3d_free_string 으로 해제. 경로 배열은 콜리 할당 2 단계(path_len→copy_path).
- 문자열은 UTF-8(한글 이름). cdecl. x64.

11.2 POD 구조체

구조체	필드
R3dStatus (enum)	OK=0 · ERR_ARG=1 · ERR_PARSE=2 · ERR_RUNTIME=3 · ERR_RANGE=4
R3dGrid	cell_mm · ox,oy,oz(origin) · nx,ny,nz(shape)
R3dParams	cell_mm-w_turn-w_clear-w_corridor-w_heur-w_heur_near-clearance_radius-clearance_connectivity-corridor_radius-rack_level_count-rack_levels[8]
R3dResult	success-length_mm-cost_mm-turns-expanded_nodes-elapsed_ms-path_len-visited_len

11.3 함수(레벨 1 — 문자열 ABI)

```
const char* r3d_version()
void r3d_free_string(char*)
R3dStatus r3d_route_scene_text(scene_text, mode, priority, out_scene_text)
    mode="multi"(순차·충돌없음)|"single"(작업별 독립). priority 늘이면 "longest".
```

11.4 함수(레벨 2 — 핸들 ABI)

```
R3dEngine* r3d_create() · void r3d_destroy(e)
R3dStatus r3d_load_scene_text(e, text)
R3dStatus r3d_set_grid(e, *g) · r3d_set_params(e, *p)
R3dStatus r3d_add_obstacle(e, minx..maxz) · r3d_add_passthrough(...)
int32_t r3d_add_task(e, sx..gz, utility, utility_group) # task index 반환
R3dStatus r3d_set_task_endpoints(e, task, sx..gz) # 인터랙티브 편집
R3dStatus r3d_set_corridor_cells(e, ijk, n) # 학습 회랑 시드(L2b)
— 라우팅 —
R3dStatus r3d_route_multi(e, priority) # 전체 순차(충돌없음)
R3dStatus r3d_route_multi_progress(e, priority, cb, user) # + 진행 콜백(뷰어 다이얼로그)
R3dStatus r3d_route_task(e, task, *out) # 단일(원본 장애물)
R3dStatus r3d_route_ripup(e, priority, max_rounds, max_ripup)
```

```

R3dStatus r3d_route_corridor(e, factor, radius) # 대형·작업별 독립
R3dStatus r3d_route_corridor_multi(e, factor, radius, priority, pipe_radius) #
대형·충돌회피
— 조회 —
R3dStatus r3d_get_result(e, task, *out)
int32_t r3d_copy_path(e, task, buf, buf_cells) # (i,j,k) 연속
int32_t r3d_copy_visited(e, task, buf, buf_cells) # 방문맵
R3dStatus r3d_set_collect_visited(e, enabled)
int32_t r3d_copy_blocked(e, buf, buf_cells) # 점유맵(buf=NULL→충수)
int32_t r3d_copy_passthrough(e, buf, buf_cells)
R3dStatus r3d_dump_scene_text(e, out_text)

```

11.5 route_multi 오케스트레이션(capi.cpp)

r3d_route_multi 는 격자 셀수에 따라 백엔드와 탐색 전략을 자동 선택한다(2 장 흐름의 구현).

판정	동작
cells = nx.ny.nz > 5,000,000	ImplicitOccupancy(복셀화 없음) + max_expansions=12,000,000
그 외(소격자·골든)	DenseOccupancy(바이트 불변) + 무제한 탐색
large_grid && !w_corridor	use_hier: 저예산 직접 A*(HIER_PROBE=300,000) 먼저
probe 소진(어려운 배관)	route_hier(coarse 가이드→fine 튜브, factor=8·radius=2)로 escalate
w_corridor>0(번들링)	회랑 시드(set_corridor_cells) + 깔린 배관 자기 회랑(Dense 경로)

11.6 진행 콜백 R3dProgressFn

r3d_route_multi_progress 가 배관마다 호출하는 cdecl 콜백. phase 0=탐색 진행(처리상태 %), 1=배관 완료(지표+경로 셀). order_index(처리 순서)·task_index(원본 인덱스)·done/total·progress01 전달. 라우팅과 같은 스레드 동기 호출(콜백 예외는 경계 금지). 뷰어 진행 다이얼로그의 실시간 표시.

12. 불변식·결정성·테스트

12.1 동결 불변식 (docs/spec)

코드	불변식
A2 / W1	A* 결정성 — (f, counter) tie-break + 고정 이웃 순서 → 동일 입력·동일 경로·확장수.
M1	다중 배관 충돌 0 — 성공 경로들은 쌍별로 셀을 공유하지 않는다(mark_pipe).
M2	원본 점유맵 불변 — 작업용 occ.copy() 사본에만 깬다.
O1	백엔드 무관 동일 결과 — Dense=Sparse=Implicit 의 좌표·점유 질의 일치.
C1	비용 가산성 — 클리어런스/회랑 페널티는 가산만(보너스 금지) → admissibility 보존.
F2 / F3 / F4	scene.txt 바이트 무손실 왕복 / None↔"" 구분 / repr(float) 동일 표기.

12.2 회귀 테스트 (ctest 10 종)

테스트	검증 내용
golden	골든 01/02/03 — Python 과 경로-expanded_nodes 정확 일치.
scene_io	scene.txt 바이트 무손실 왕복(F2~F4).
occupancy	Dense/Sparse 점유 질의 일치.
implicit	Dense==Implicit is_blocked 전수 일치 + 거대격자 오버플로/OOM 없음.
corridor	계층 corridor 가 전역 최단과 동일 길이(전체 허용 시 astar 동일).
ripup	rip-up 이 혼잡 해소(합성 1/2→2/2). Python 동일 값.
attract	회랑(w_corridor) 소프트 바이어스 동작.
capi	DLL 경유 골든 03 5/5 + 문자열 왕복.
vdb	OpenVDB 백엔드(선택 빌드) 일치.
fcl	FCL 정밀 충돌(틈 200mm 가는/굵은 파이프 구별).

12.3 실데이터 교차검증 (Python = C++ = C#)

project6_c100(cell=100, 장애물 983·작업 208): 194/208 · 3,400,800mm 3 자 완전 일치.

project6(cell=200): multi 77 / ripup 80(+3) — rip-up 실데이터 개선 실측. 합성 혼잡(9×9 벽+틈 2): seq 1/2 → ripup 2/2(C++/Python 동일).

13. 주요 변수·파라미터 종합 참조

13.1 비용 파라미터(RouteParams / R3dParams)

변수	기본	효과 / 튜닝
cell_mm	50	셀 한 번·직진 1 칸 비용. 작을수록 정밀·탐색량 ↑ (25/10mm 는 Implicit).
w_turn	500	꺾임 가산. 클수록 직선 선호.
w_clear	10	장애물 근접 회피 강도.
clearance_radius	2	근접 페널티 범위(셀).
w_heur	1.0 (뷰어 2.0)	1=최적(골든), >1=목표지향·확장 급감·약간 비최적.
w_heur_near	0.0 (뷰어 1.0)	동적 가중 목표근처 값 → 근처 정확·먼곳 빠름.
w_corridor	0.0	>0=기존설계 유사 번들링(회랑 밖 가산). 뷰어 cell:0.2~0.5.
corridor_radius	1	회랑 성장/시드 반경.
rack_levels[]	—	선호 단(z 셀) → 회랑 면제(주 배관랙 유도).

13.2 라우팅 제어 변수

변수	의미
priority	작업 순서: longest(기본)-shortest-utility-original.
pipe_radius	깔린 배관 점유 팽창 반경(셀). 0=경로 셀만.
snap_to_free	종단 점유 시 자유셀 탐색 반경(셀, 기본 2).
max_expansions	탐색 상한(-1=무제한). 대격자 자동 12,000,000.
max_rounds / max_ripup	rip-up 라운드 상한 / 한 번에 뜯는 배관 수 상한.
collect_visited	방문 셀 수집 on/off(가시화·메모리).

13.3 거대격자 오케스트레이션 상수(capi.cpp)

상수	값	의미
백엔드 전환 임계	5,000,000 셀	초과 시 Dense→Implicit.
max_exp(대격자)	12,000,000	대격자 탐색 상한.
HIER_PROBE	300,000	저예산 직접 A* — 초과 시 hier escalate.
HIER_FACTOR	8	계층 corridor coarse/fine 셀 비율.

HIER_RADIUS	2	corridor 튜브 팽창 반경(coarse 셀).
-------------	---	------------------------------

이상으로 C++ 라우팅 엔진의 전체 프로세스·핵심 알고리즘·주요 함수·변수를 정리했다. 더 깊은 세부는 각 헤더의 한글 주석과 ctest 가 정답이며, Python 레퍼런스(routing3d_py)가 알고리즘 동등성의 기준이다.